

CUDA API Libraries

Parallel and Heterogeneous Computing

David Mayerich

Scalable Tissue Imaging and Modeling (STIM) Laboratory

Department of Electrical and Computer Engineering

Cullen College of Engineering

University of Houston

UNIVERSITY of **HOUSTON** | ECE

Standard Libraries

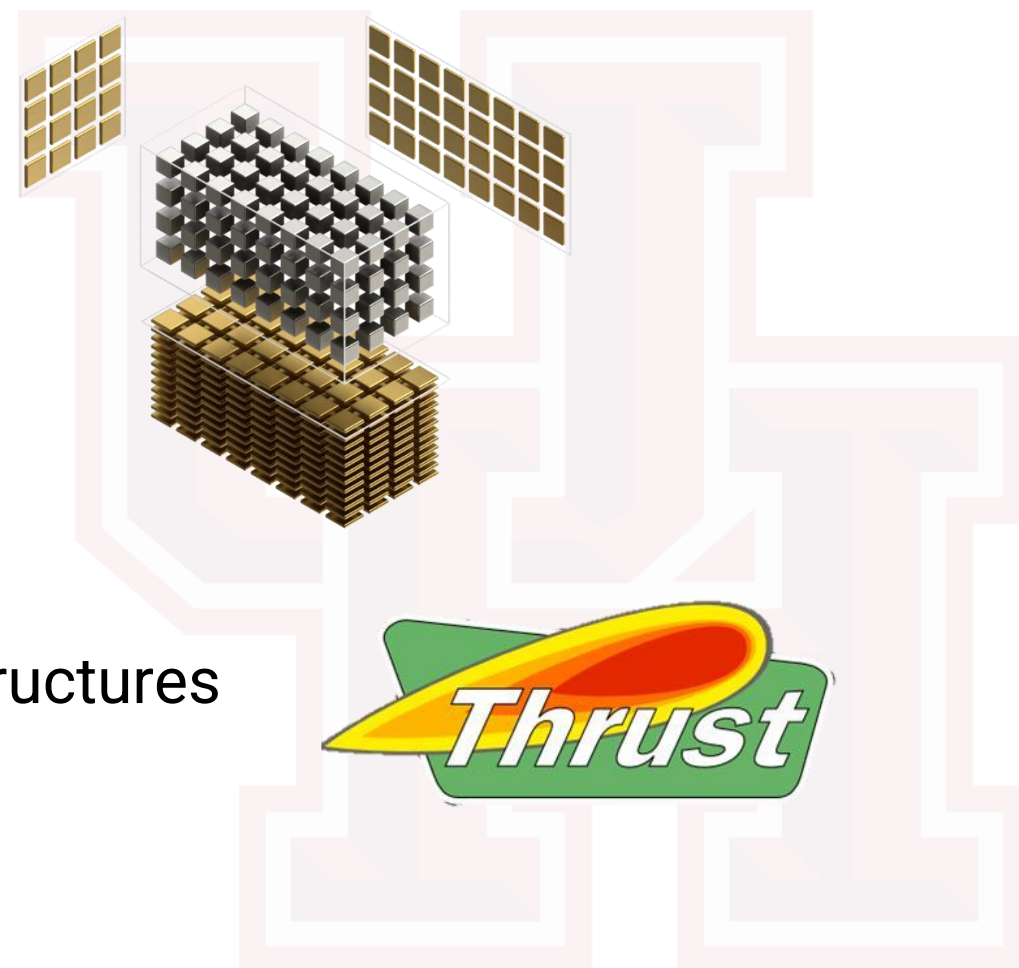
API-Level

- **cuBLAS** – basic linear algebra subroutines
- **cuFFT** – fast Fourier transform
- **cuSOLVER** – dense and sparse direct solvers
- **cuSPARSE** – sparse matrix subroutines
- **NPP** – image and video processing primitives
- **nvGRAPH** – graph analytics library

- **Thrust** – templated parallel algorithms and data structures

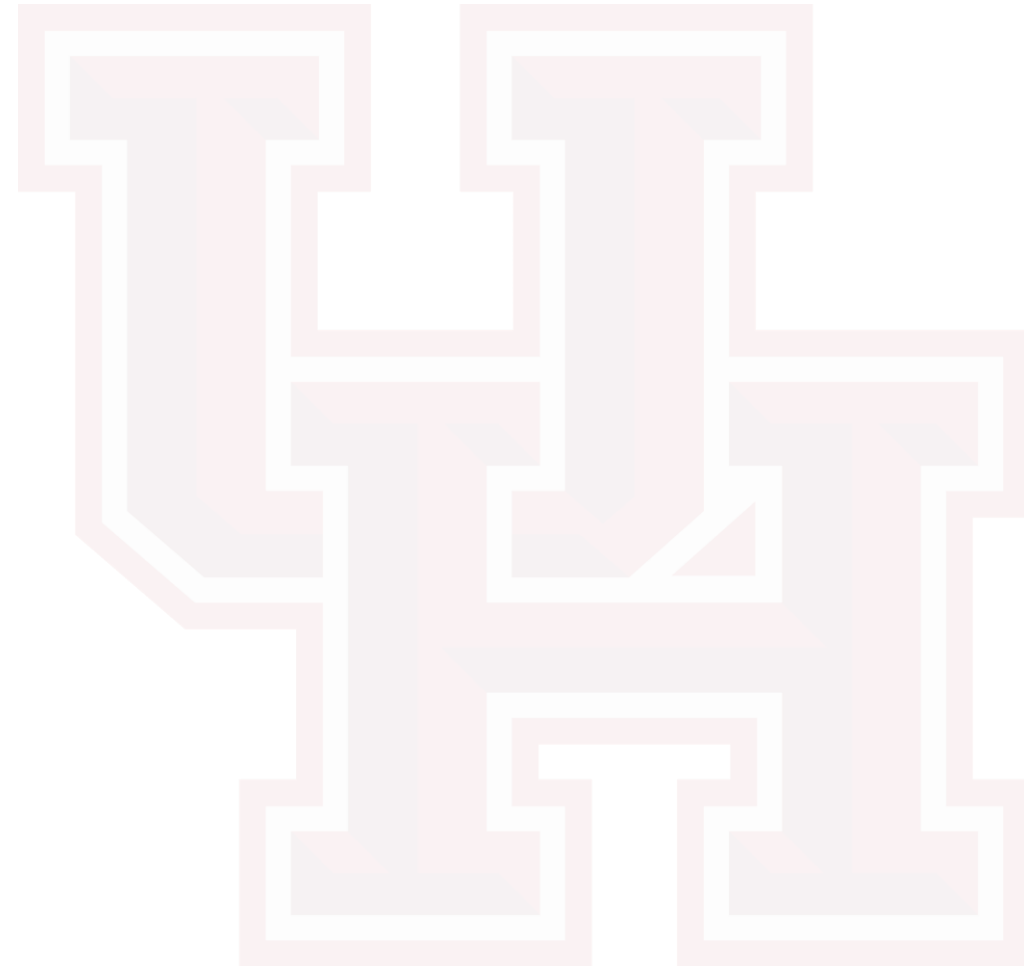
Kernel-Level

- **cuRAND** – GPU random number generator



cuBLAS

BLAS – Basic Linear Algebra Subroutines



BLAS – Basic Linear Algebra Subprograms

- Specification for low-level linear algebra routines
 - Implementations: OpenBLAS, Intel MKL, ...
 - Uses: MATLAB, NumPy, GNU Octave, ...
 - netlib Reference Implementation (FORTRAN): <https://www.netlib.org/blas/>
- Functions are specified with a prefix and function name
 - (s) **single** (32-bit) precision
 - (d) **double** (64-bit) precision
 - (c) single **complex** (total = 64 bits)
 - (z) double **complex** (total = 128 bits)
 - common function names: *axpy, *gemv, *gemm, ...
 - saxpy() – add two single-precision (32-bit) vectors
 - zgemm() – multiply complex double-precision matrices

BLAS Subroutine Levels

- Level 1 – vector/vector operations

- “a x plus y” (axpy): $\mathbf{y} \leftarrow \alpha \mathbf{x} + \mathbf{y}$

- dot product (dot): $d \leftarrow \mathbf{x}^T \mathbf{y}$

- “absolute value sum” (asum): $a \leftarrow \|\operatorname{Re}\{\mathbf{x}\}\|_1 + \|\operatorname{Im}\{\mathbf{x}\}\|_1$

- Level 2 – matrix/vector operations

- “generalized matrix-vector multiplication” (gemv): $\mathbf{y} \leftarrow \alpha \mathbf{A} \mathbf{x} + \beta \mathbf{y}$

- “triangular packed solver” (tpsv): $\mathbf{x} \leftarrow \mathbf{A}^{-1} \mathbf{x}$

- Level 3 – matrix/matrix operations

- “general matrix multiplication” (gemm): $\mathbf{C} \leftarrow \alpha \mathbf{A} \mathbf{B} + \beta \mathbf{C}$

CUDA Basic Linear Algebra Subprograms

- `cudaError_t cublasCreate(cublasHandle_t* h)`
 - initialize cuBLAS and returns a handle containing cuBLAS device information
 - use the `cublasHandle_t` value to execute cuBLAS functions
- `cudaError_t cublasDestroy(cublasHandle_t h)`
 - destroy a cuBLAS handle and free any required resources

```
float* x = (float*) malloc(N * sizeof(float)); //allocate host arrays
...                                           //initialize arrays

cublasHandle_t handle;
cublasCreate(&handle);                      //initialize cuBLAS

float* gpu_x;
cudaMalloc(&gpu_x, N * sizeof(float));
cudaMemcpy(gpu_x, x, N * sizeof(float));    //copy array to the device
...                                           //call cuBLAS functions
```

Calling cuBLAS Functions

- function calls use device pointers – copy data to the GPU first
- functions that return scalar values (min, max, sum, etc.) can accept a host pointer for the return value
 - basically, you don't have to copy a scalar value from the device
- cuBLAS calls follow a predictable format
- a character identifier in the function signature indicates data type
 - s – single precision (32-bit) real
 - d – double precision (64-bit) real
 - c – single precision complex (2 x 32-bit values per element)
 - z – double precision complex

Level 1 – Index Functions

- Functions that return indices to array elements

`cublasI<t>a-----()`

`cublasIsamin(cublasHandle_t h, int n, const float* x, int incx, int* result)`

- returns the index to the smallest i such that $|x_i| \leq \forall x \in \mathbf{x}$
- incx specifies the stride, or spacing between array elements (usually 1)

- **Indices are 1-based, following FORTRAN convention!!!**

```
float* x = (float*) malloc(N * sizeof(float));
```

```
...
```

```
//load array elements
```

```
float* gpu_x;
```

```
cudaMalloc(&gpu_x, N * sizeof(float));
```

```
//allocate device memory
```

```
cudaMemcpy(gpu_x, x, N * sizeof(float), cudaMemcpyHostToDevice);
```

```
int i;
```

```
cublasIsamin(handle, N, gpu_x, 1, i);
```

```
//call the cuBLAS function
```

Level 1 – Index Functions

- CUDA defines `cuComplex` and `cuDoubleComplex` for simplicity
 - both are two-element structures
- Find the largest value $|x_i| \in \mathbf{x}$ where $\mathbf{x} \in \mathbb{C}^N$

```
cuComplex* x = (cuComplex *) malloc(N * sizeof(cuComplex));
...
//load array elements

float* gpu_x;
cudaMalloc(&gpu_x, N * sizeof(cuComplex)); //allocate device memory
cudaMemcpy(gpu_x, x, N * sizeof(cuComplex), cudaMemcpyHostToDevice);
int i;
cublasIcamin(handle, N, gpu_x, 1, i); //call the cuBLAS function
```

Level 1 – Scalar Functions

`cublas<t>a----()`

- For `<t>`:
 - S – single precision real
 - D – double precision real
 - Sc – single precision complex
 - Dz – double precision complex

`cublasSasum(cublasHandle_t h, int n, const float* x, int incx, float* y)`

`cublasDzasum(cublasHandle_t h, int n, const cuDoubleComplex* x, int incx, double* y)`

$$y = \sum_{i=1}^N |x_i|$$

- Note that `y` can be a host or device pointer
 - cuBLAS will automatically detect the location

Level 3 – General Matrix Multiplication

`cublas<t>gemm()`

- `<t>` = 'H' offers support for half-precision (16-bit) floating point

$$\mathbf{C} = \alpha \cdot p_a(\mathbf{A}) \cdot p_b(\mathbf{B}) + \beta \mathbf{C}$$

```
cublasSgemm( cublasHandle_t h,           //cuBLAS handle
             cublasOperation_t transa,    //operation performed on A
             cublasOperation_t transb,    //operation performed on B
             int m, int n, int k,         //A = m x k, B = k x n, C = m x n
             const float* alpha,          //optional factor for A*B
             const float* A, int lda,     //A data and first dimension size
             const float* B, int ldb,     //B data and first dimension size
             const float* beta,           //optional scalar factor for C
             float* C, int ldc            //C data and first dimension size
);
```

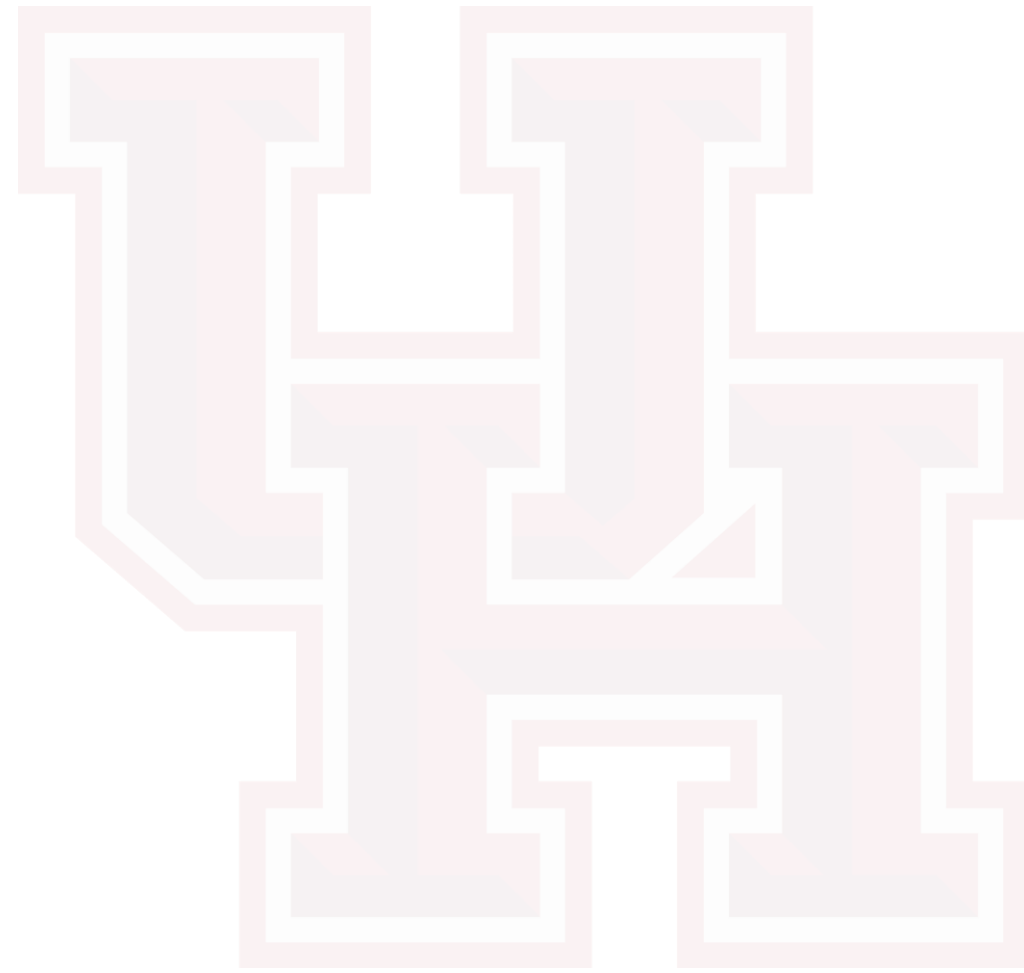
Level 3 – General Matrix Multiplication

- gemm allows an operation to be applied to each input matrix:
 - CUBLAS_OP_N do nothing
 - CUBLAS_OP_T transpose the matrix
- Calculate $C = AB$

```
float* A = (float*) malloc(M * K * sizeof(float));
float* B = (float*) malloc(K * N * sizeof(float));
float* C = (float*) malloc(M * N * sizeof(float));
...                               //load arrays, generate handle, copy to device
float alpha = 1.0;               //no scalar factors are applied
float beta = 1.0;
cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N,
            M, N, K, &alpha,
            gpu_A, m, gpu_B, k, &beta, gpu_C, m);
cudaMemcpy(C, gpu_C, M * N * sizeof(float), cudaMemcpyDeviceToHost);
```

cuSOLVER

LAPACK – Linear Algebra PACKage



LAPACK – Linear Algebra PACKage

- Specification for higher-level linear algebra routines
 - Commonly used with BLAS: Accelerate (Apple), Intel MKL, SciPy, ...
 - netlib Reference Implementation (FORTRAN): <https://www.netlib.org/lapack/>
- Similar naming convention (prefix + function)
- Addition of a two-letter “matrix code” identifying the types of matrices provided:
 - bd = bidiagonal matrix
 - ge = general matrix
 - sy = symmetric matrix
 - tr = triangular matrix

Solve $AX = B$ with 32-bit precision

LAPACK FORTRAN Standard (netlib.org)

sgesv(

integer

n,



matrix size



integer

nrhs,



number of
right-hand-sides



real

a,



matrix A



integer

lda,



leading A
dimension



integer

ipiv,



pivot indices



real

b,



matrix B



integer

ldb,



leading B
dimension



integer

info)



0 = success

LAPACK C Bindings (Intel oneAPI)

LAPACK_sgesv(int layout,

int

n,

int

nrhs,

float* a

int

lda,

int* ipiv,

float* b,

int

ldb)

Solve $AX = B$ with 32-bit precision (cuSolver)

cusolverDn **S** **S** **gesv**
cusolver dense precision function

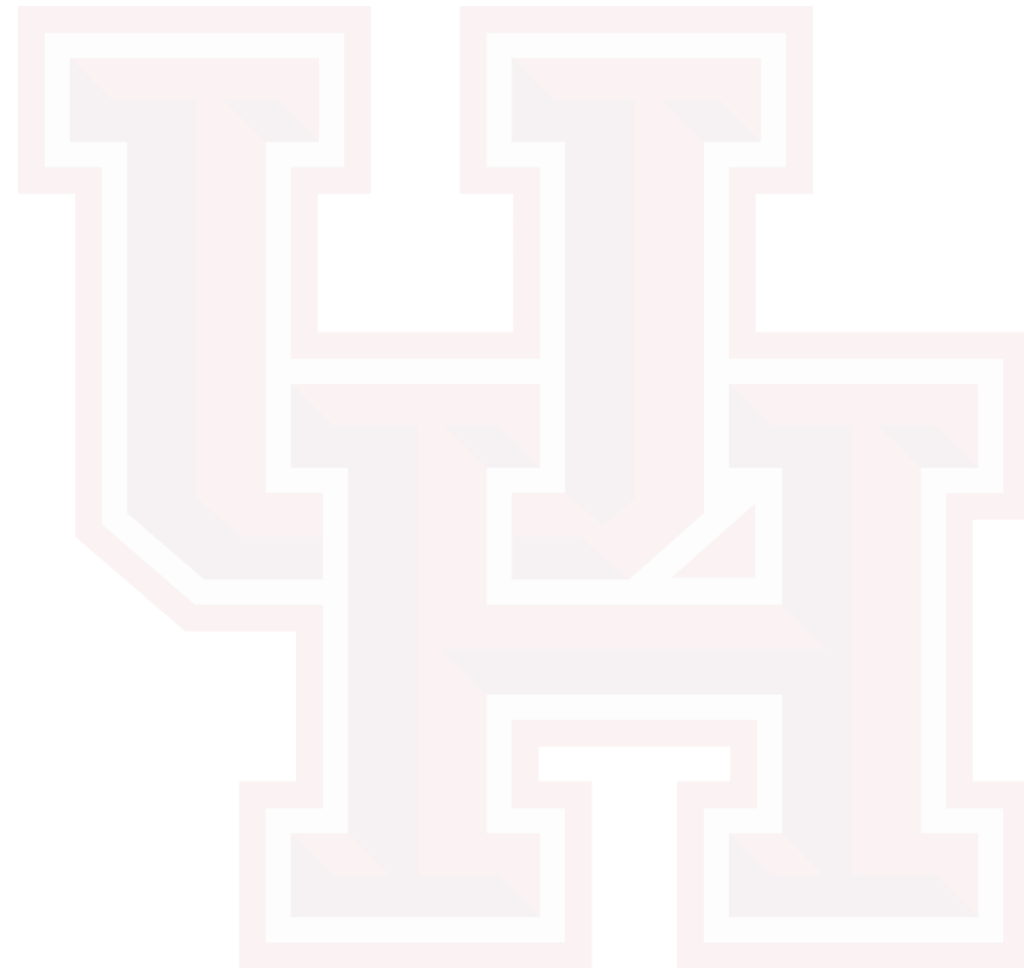
```
cusolverStatus_t cusolverDnSSgesv(  
    cusolverDnHandle_t handle, ← cusolverDN context  
    int n, ← number of rows in  $A$   
    int nrhs, ← right-hand-sides to solve  
    float* dA, ← matrix  $A$   
    int ldda, ← leading dimension of  $A$   
    int* dipiv, ← permutation vector  
    float* dB, ← matrix  $B$   
    int lddb, ← leading dimension of  $B$   
    float* dX, ← matrix  $X$   
    int lddx, ← leading dimension of  $X$   
    void* dWorkforce, ← workspace device pointer  
    size_t lwork_bytes, ← workspace size  
    int* niter ← error output  
    int* dinfo) ← solver status
```

Workspace Allocation

```
cusolverStatus_t cusolverDnSSgesv_bufferSize(  
    cusolverHandle_t handle,  
    int n,  
    int nrhs,  
    float* dA,  
    int ldda,  
    int* dipiv,  
    float* dB,  
    int lddb,  
    float* dX,  
    int lddx,  
    void* dwork,  
    size_t* lwork_bytes);
```

cuFFT

FFTW – Fastest Fourier Transform in the West



Discrete Fourier Transform

- DFT definition – $O(n^2)$

$$X_v = \sum_{t=0}^{T-1} x_t e^{\frac{-2\pi i}{T} nt}$$

$$v = [0, \dots, N - 1]$$

- Fast Fourier Transform (FFT) – $O(n \log n)$
 - Cooley – Turkey algorithm
 - recursively divide the FFT into blocks of size $N/2$

FFTW – Fastest Fourier Transform in the West

- Fastest Fourier Transform in the West (FFTW)
 - fastest open-source FFT (used in MATLAB)
 - developed by Frigo and Johnson (MIT)
 - uses Cooley – Turkey
 - written in C: <http://www.fftw.org/>

```
#include<fftw3.h>
...
fftw_complex* f = (fftw_complex*) malloc(N * sizeof(fftw_complex));
fftw_complex* g = (fftw_complex*) malloc(N * sizeof(fftw_complex));
    //create an FFTW handle (allocate FFTW data and resources)
fftw_plan p = fftw_plan_dft_1d(N, f, g, FFTW_FORWARD, FFTW_ESTIMATE);
fftw_execute(p);                //perform the DFT
fftw_destroy_plan(p);           //free memory for the fftw handle
...                             //use result, free memory...
```

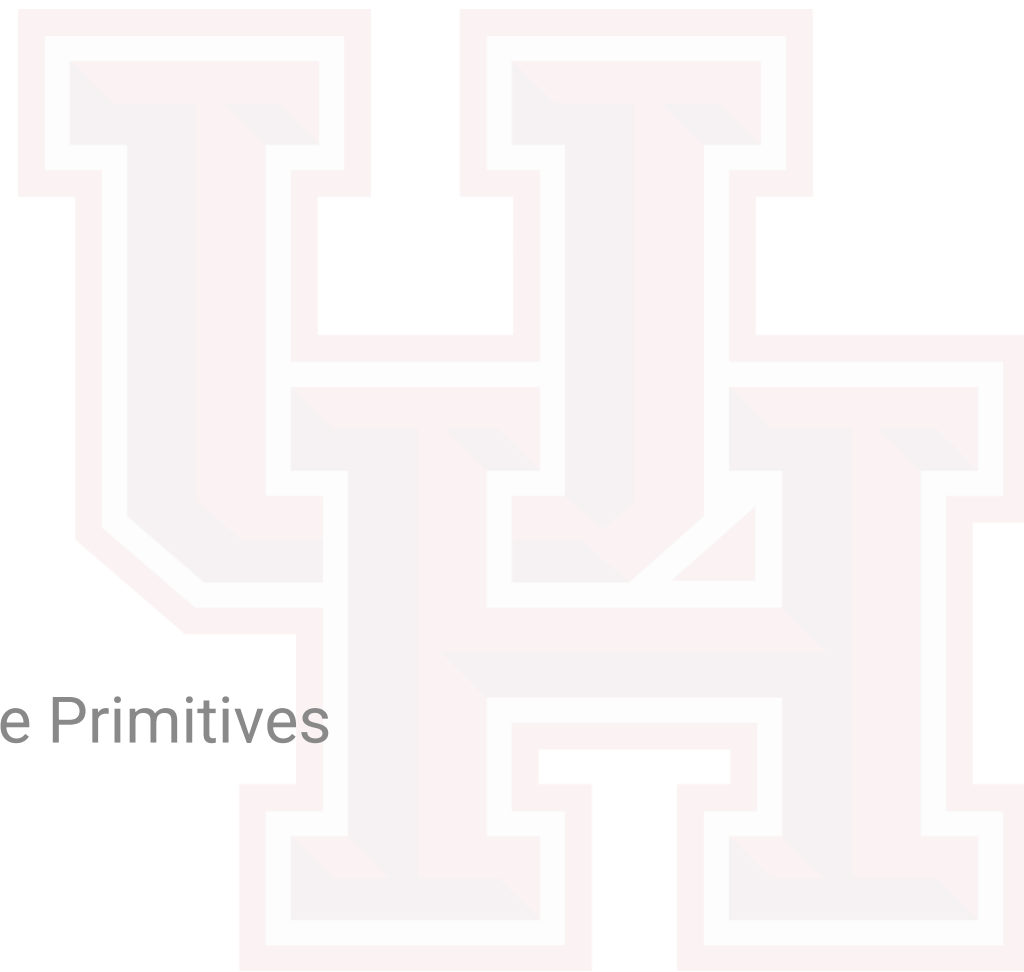
cuFFT

- CUDA DFT implementation

```
#include< cufft.h>
...
cufftComplex* f = (cufftComplex*) malloc(N * sizeof(cufftComplex));
...
//load data
cufftHandle h; //create a cuFFT handle
cufftComplex* gpu_f; //create a device pointer
cudaMalloc(&gpu_f, N * sizeof(cufftComplex)); //allocate device space memory
cudaMemcpy(gpu_f, f, N * sizeof(cufftComplex), cudaMemcpyHostToDevice);
cufftPlan1d(&h, N, CUFFT_C2C, 1); //allocate cuFFT resources
cufftExecC2C(h, gpu_f, gpu_f, CUFFT_FORWARD); //perform the DFT
...
//use the data, free memory...
```

NPP

NVIDIA 2D Image and Signal Processing Performance Primitives



NPP Functions

Filtering Examples

- Image Convolution
- Computer Vision
 - Distance Transform
 - Canny Edge Detection
 - Hough Transform
- Flood Fill
- Connected Components

nppiFilterCannyBorder_8uC1R_Ctx(...)

```
NppStatus nppiFilterCannyBorder_8u_C1R_Ctx(const Npp8u *pSrc, int nSrcStep, NppiSize oSrcSize, NppiPoint oSrcOffset, Npp8u *pDst, int nDstStep, NppiSize oSizeROI, NppiDifferentialKernel eFilterType, NppiMaskSize eMaskSize, Npp16s nLowThreshold, Npp16s nHighThreshold, NppiNorm eNorm, NppiBorderType eBorderType, Npp8u *pDeviceBuffer, NppStreamContext nppStreamCtx)
```

1 channel 8-bit unsigned grayscale to 1 channel 8-bit unsigned black (0x00) and white (0xFF) image with border control.

Parameters:

- > **pSrc** – Source-Image Pointer.
- > **nSrcStep** – Source-Image Line Step.
- > **oSrcSize** – Source image width and height in pixels relative to pSrc.
- > **oSrcOffset** – The pixel offset that pSrc points to relative to the origin of the source image.
- > **pDst** – output edge destination_image_pointer.
- > **nDstStep** – output edge destination_image_line_step.
- > **oSizeROI** – Region-Of-Interest (ROI).
- > **eFilterType** – selects between Sobel or Scharr filter type.
- > **eMaskSize** – fixed filter mask size to use.
- > **nLowThreshold** – low hysteresis threshold value.
- > **nHighThreshold** – high hysteresis threshold value.
- > **eNorm** – gradient distance method to use.
- > **eBorderType** – source image border type to use.
- > **pDeviceBuffer** – pointer to scratch DEVICE memory buffer of size hpBufferSize (see `nppiFilterCannyBorderGetBufferSize()` above)
- > **nppStreamCtx** – Application Managed Stream Context.

Returns: Image Data Related Error Codes, ROI Related Error Codes